GUIDE 25 · AGENTS

Let an agent explore your schema and run ad-hoc SQL

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=25-mcp-ad-hoc-tools. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** an AI agent that can look at a database's tables and ask its own read-only questions with SQL - limited to the connections and tables you choose, and recorded like every other run.

Saved queries are the safest thing to give an agent: you wrote them, and they take typed parameters. But an agent exploring data will ask questions nobody saved a query for. Two built-in MCP tools cover that:

TOOL	DOES	SAME AS REST
`list_tables`	lists a connection's tables and views, with columns, types and keys	`GET /connections/<name>/schema`
`execute_sql`	runs one **read-only** SQL statement, with bound parameters	`POST /execute_sql`

This guide assumes the MCP server from Let an AI agent call your saved queries via MCP is running.

Step 1: A key with connection access

The two tools appear only for a key with a `connections` grant - the same grant that allows ad-hoc SQL over REST. A key granted only `queries` or `collections` never sees them (verified: a collection-only key listed its three query tools and nothing else). Narrow it to the tables the agent needs:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "agent", "connections": ["shop"], "allowed_tables": ["orders", "customers", "products"]
}'
```

Step 2: What the agent sees

`tools/list` for this key, verified:

```text
['orders_by_status', 'salary_list', 'list_tables', 'execute_sql']
```

The saved queries on `shop` first (from the `connections` grant), then the two ad-hoc tools. Their input schemas:

```json
{"name": "list_tables", "inputSchema": {"type": "object", "required": ["connection_name"], "properties":
{
   "connection_name": {"type": "string", "description": "A connection name this key is permitted to
use."},
   "database": {"type": "string", "description": "Browse a different database on the same server - admin
key only."}}}}

{"name": "execute_sql", "inputSchema": {"type": "object", "required": ["connection_name", "sql"],
"properties": {
   "connection_name": {"type": "string", ...},
   "sql": {"type": "string", "description": "A single SELECT/WITH/SHOW/DESCRIBE/EXPLAIN statement."},
   "params": {"type": "object", "description": "Values for any :name bound parameters in the SQL."},
   "page_size": {"type": "integer", "description": "Row cap, up to 200 (the server default)."},
   "database": {"type": "string", ...}}}}
```

Step 3: Calls, and what comes back

With the `mcp` Python SDK (any MCP client works the same way):

```python
await session.call_tool("list_tables", {"connection_name": "shop"})
# structuredContent: {"tables": [{"name": "customers", "schema": "main", "type": "table", "columns": [
#     {"name": "id", "type": "INTEGER", "nullable": true, "position": 1, "primary_key": true,
"foreign_key": null}, ...]},
#   ...], "truncated": false}

await session.call_tool("execute_sql", {"connection_name": "shop",
    "sql": "SELECT * FROM orders WHERE total > :min", "params": {"min": 20}})
# structuredContent: {"rows": [{"id": 1, "customer_id": 1, "total": 42.5, "status": "paid"},
#                              {"id": 2, "customer_id": 2, "total": 87.25, "status": "paid"}],
"truncated": false}
```

`:min` is a real bound parameter, exactly as over REST - encourage agents to pass values in `params` rather than pasting them into the SQL text.

What's refused

Each of these came back as a tool error (`isError: true`) with a `code` in `structuredContent`, verified:

CALL	`CODE`	WHY
`DELETE FROM orders`	`read_only`	`execute_sql` is **always** read-only over MCP - even for a key with `allow_writes`, even with `QUERYAPIGATE_ALLOW_WRITES` on. (The message suggests setting `QUERYAPIGATE_ALLOW_WRITES`; over MCP that doesn't change anything.)
`SELECT * FROM salaries`	`table_not_allowed`	`allowed_tables` applies to MCP as to REST
`SELECT 1; SELECT 2`	`multiple_statements`	one statement per call
any SQL on `examples`	`connection_forbidden`	not in the key's `connections`

An agent can read the `code` and adjust - ask for different tables, drop the write - rather than parse the message.

Keep in mind

- **`list_tables` shows only the tables `allowed_tables` permits** - for this key `customers`, `orders` and `products`, not `salaries`. Saved-query tools are listed by the key's grants alone, though: `salary_list` appears above, and fails with `table_not_allowed` when called. Keep a query the agent mustn't see out of its collections.
- **Results are capped** at `QUERYAPIGATE_MCP_MAX_ROWS` rows (default 200) - see Read structured MCP results properly for `truncated` and `page_size`.
- **Everything is recorded.** Each `execute_sql` call is an ad-hoc run in run history with `"transport": "mcp"`, the key name, the SQL (as `QUERYAPIGATE_HISTORY_ADHOC_SQL` allows) and the names - not values - of its parameters:

```shell
curl 'http://127.0.0.1:5000/api/v1/history?kind=adhoc' -H 'X-API-Key: demo-key'
# {"items": [{"kind": "adhoc", "key_name": "agent", "transport": "mcp", "connection_name": "shop",
#             "status": "error", "code": "multiple_statements", "sql": "SELECT 1; SELECT 2", ...},
...]}
```

- **Use a database user with read rights only.** The SQL check is one layer; a read-only database user is the other. On PostgreSQL, MySQL, ClickHouse and SQLite the session is also read-only at the database level (see the support matrix).
- **A timeout protects the database** from an agent's runaway query: `QUERYAPIGATE_QUERY_TIMEOUT` (default 30 seconds) applies to MCP calls too.

Next steps

- Read structured MCP results properly.
- Restrict a key to specific tables.
- Check the MCP server from the Console.